
TP Cellule Robotisée - Robot Stäubli TX60L

Realisé par :

- Aya Briksine

Sous la direction de :

- M.ELAHRES

Table des matières

Introduction	3
1 Présentation de la cellule robotisée	5
1.1 Le robot Stäubli TX60L — structure mécanique et caractéristiques . .	5
1.2 Les éléments de la cellule robotisée	6
1.3 Volume de travail du robot	6
2 Analyse de la cellule robotisée	8
2.1 Schéma bloc de la cellule	8
2.2 Flux informationnels entre les sous-systèmes	9
2.3 Commande d'un axe	9
3 Mise en œuvre et programmation	10
3.1 Procédure de mise en marche du robot	10
3.1.1 Phase de préparation	10
3.1.2 Phase d'utilisation en mode manuel	10
3.1.3 Phase d'utilisation en mode programmé	10
3.1.4 Phase d'arrêt du robot	11
3.2 Étude de la zone de sécurité et arrêt d'urgence	11
3.3 Prise en main en mode manuel (Teach)	11
3.4 Relevé des coordonnées cartésiennes	12
3.5 Test d'un programme simple en mode automatique	12
3.6 Programme final de transfert A → B en VAL 3	13
3.7 Visualisation de la trajectoire calculée	16
4 Résultats et observations	17
4.1 Comportement du robot lors de l'exécution	17
4.2 Difficultés rencontrées et solutions apportées	18
Conclusion	20

INTRODUCTION

Dans le cadre de notre formation en génie mécanique et automatisme, nous avons réalisé un travail pratique portant sur une **cellule robotisée** équipée d'un robot industriel **Stäubli TX60L** à 6 axes. Ce TP s'inscrit dans une démarche pédagogique visant à mettre en pratique les méthodes d'analyse et de conception des systèmes automatisés, ainsi qu'à établir un dialogue concret entre la **partie opérative** et la **partie commande**.

L'objectif principal de ce travail est double : d'une part, analyser la structure mécanique et fonctionnelle de la cellule robotisée, et d'autre part, programmer une tâche de manipulation consistant à transférer des objets d'un point **A** vers un point **B** à l'aide du robot.

La cellule robotisée étudiée est composée de plusieurs éléments interconnectés qui forment un système automatisé cohérent. Comme illustré sur la Figure 1, le système s'articule autour de cinq sous-systèmes principaux. Le **PC équipé du logiciel VAL 3** constitue l'interface de programmation ; il communique avec l'**armoire de commande CS8** via une liaison Ethernet. Le **boîtier TEACH** permet une commande manuelle directe du robot pour l'apprentissage des points de passage. L'armoire de commande pilote les six axes du **robot Stäubli TX60L** et récupère en retour les informations des encodeurs. Enfin, le robot agit sur l'environnement via sa **pince pneumatique**, alimentée entre 1,5 et 2 bars.

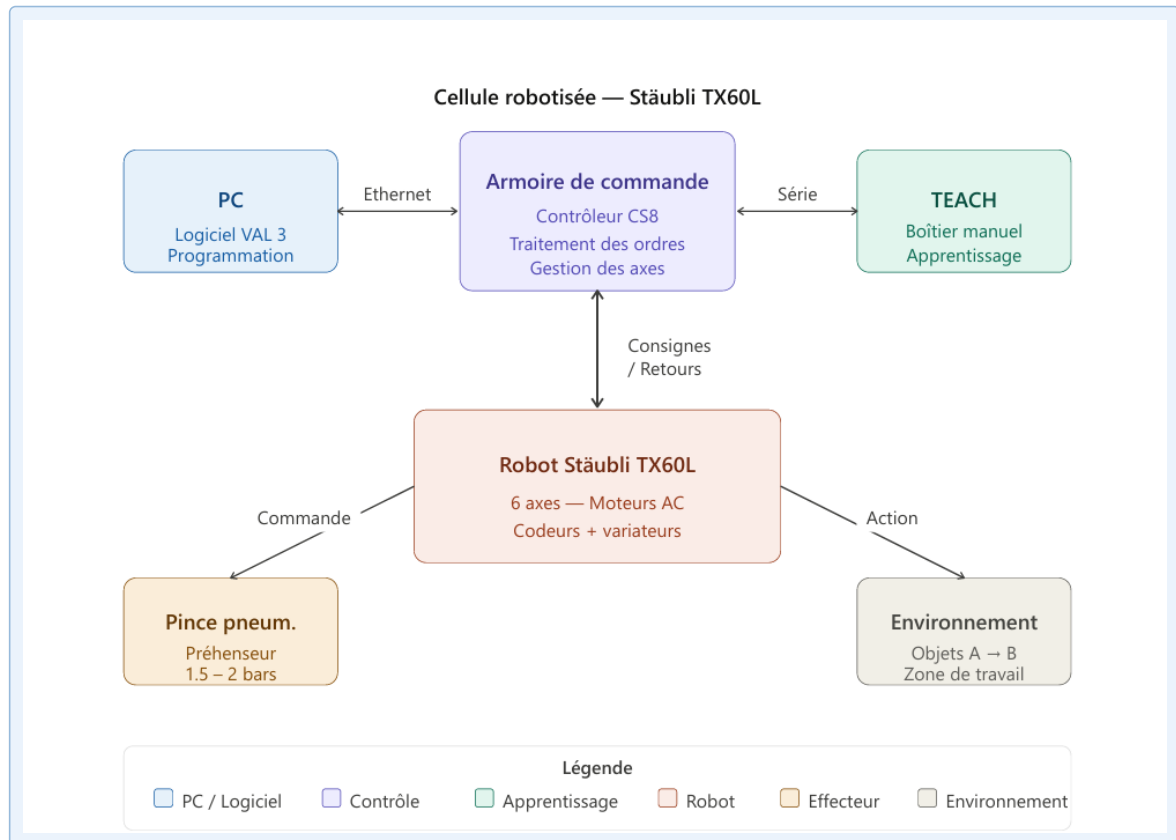


Figure 1. Schéma bloc de la cellule robotisée Stäubli TX60L

Ce rapport est organisé en deux grandes parties : la première est consacrée à l'**analyse du système**, et la seconde présente la **programmation et la manipulation pratique** du robot.

PRÉSENTATION DE LA CELLULE ROBOTISÉE

1.1 Le robot Stäubli TX60L — structure mécanique et caractéristiques

Le robot Stäubli TX60L est un robot manipulateur industriel à **6 axes de rotation**, appartenant à la famille des robots à architecture série (polyarticulé). Il est monté sur une base fixe et dispose d'une chaîne cinématique ouverte composée de six liaisons rotoïdes, ce qui lui confère une grande souplesse de mouvement dans son espace de travail.

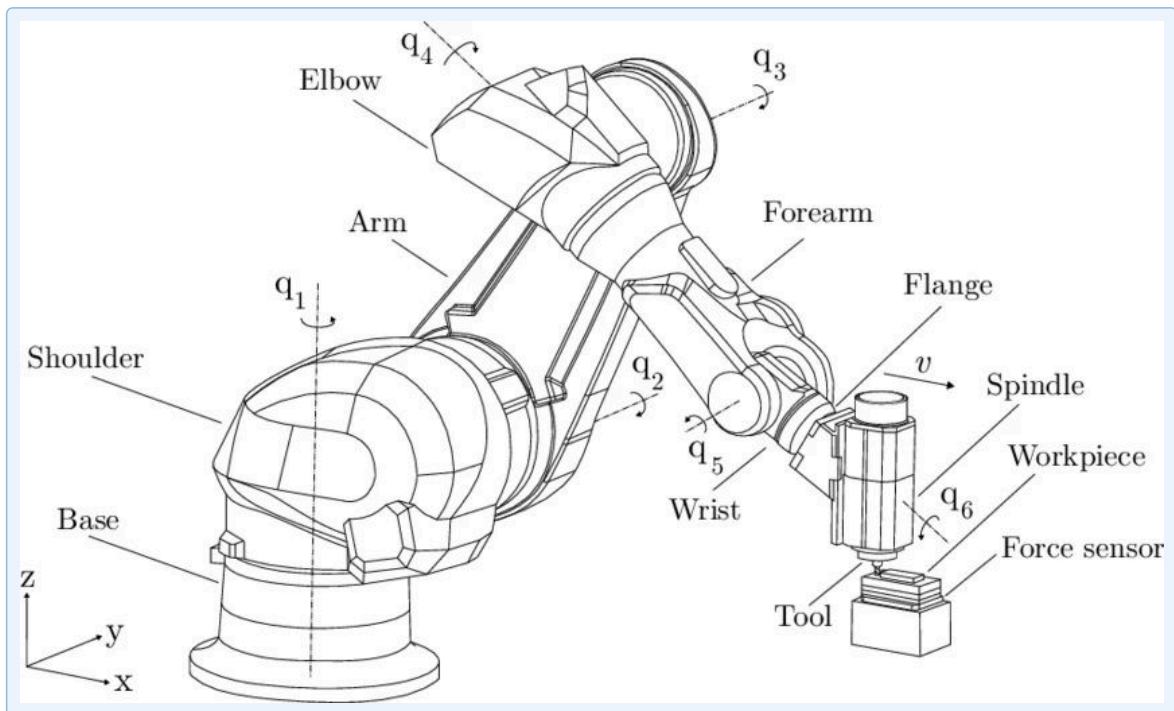


Figure 1.1. Vue d'ensemble du robot Stäubli TX60L avec identification des axes J1–J6

Paramètre	Valeur
Nombre d'axes	6 (J1 à J6)
Charge maximale	6 kg
Portée maximale	≈ 920 mm (version L — <i>Long reach</i>)
Répétabilité	± 0,02 mm
Motorisation	Moteurs électriques CC avec encodeurs

1.2 Les éléments de la cellule robotisée

La cellule robotisée mise à notre disposition est composée des éléments suivants :

- **Le robot Stäubli TX60L** : la partie opérative, chargée d'exécuter les mouvements physiques.
- **L'armoire de commande** : contient le contrôleur qui gère le calcul des trajectoires et la commande des moteurs.
- **Le boîtier Teach** : interface de programmation manuelle permettant de déplacer le robot axe par axe et d'enregistrer des points.
- **Le PC avec VAL 3 Studio** : environnement de développement permettant d'écrire, compiler et télécharger les programmes vers le contrôleur via une liaison Ethernet.

1.3 Volume de travail du robot

Le volume de travail du Stäubli TX60L correspond à l'ensemble des positions atteignables par l'effecteur. Pour la version TX60L, la portée étendue offre un espace de travail sphérique d'environ **920 mm de rayon**, avec une zone morte au voisinage immédiat de la base du robot.

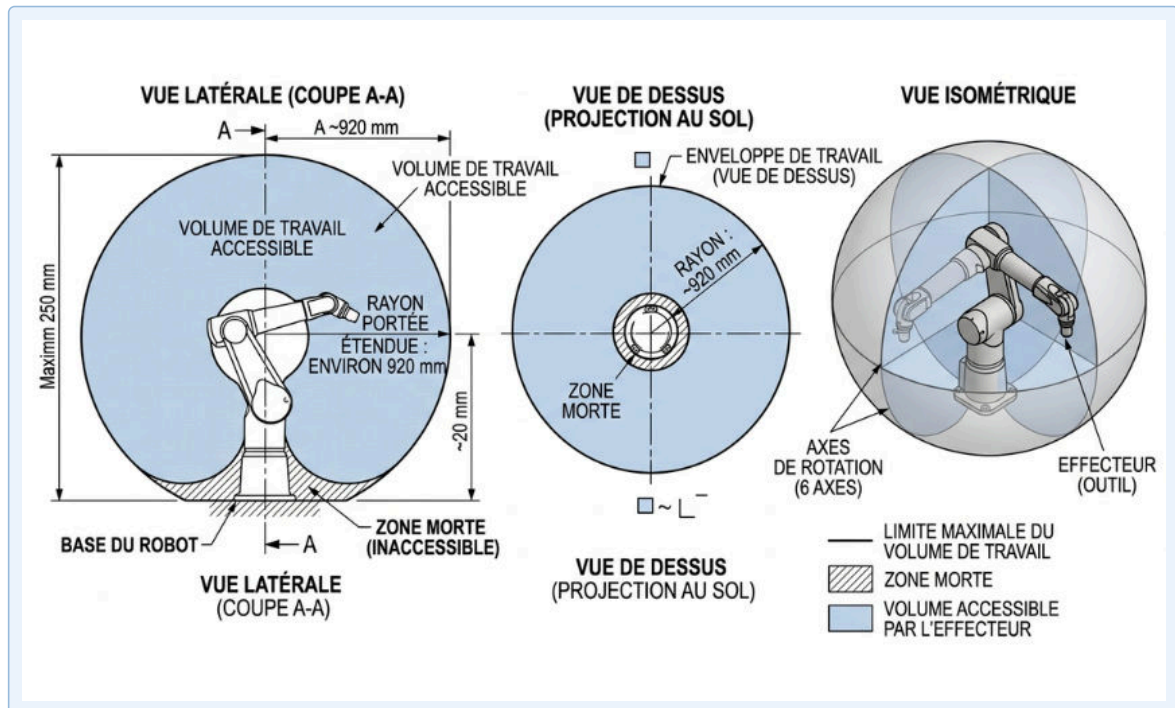


Figure 1.2. Volume de travail du Stäubli TX60L

ANALYSE DE LA CELLULE ROBOTISÉE

2.1 Schéma bloc de la cellule

La cellule robotisée peut être décomposée en plusieurs sous-systèmes en interaction. L'opérateur interagit avec le PC équipé de VAL 3 Studio pour programmer les tâches, le programme est transmis à l'armoire de commande via Ethernet, et le boîtier Teach permet une interaction manuelle directe. L'armoire pilote le robot et reçoit en retour les positions des encodeurs.

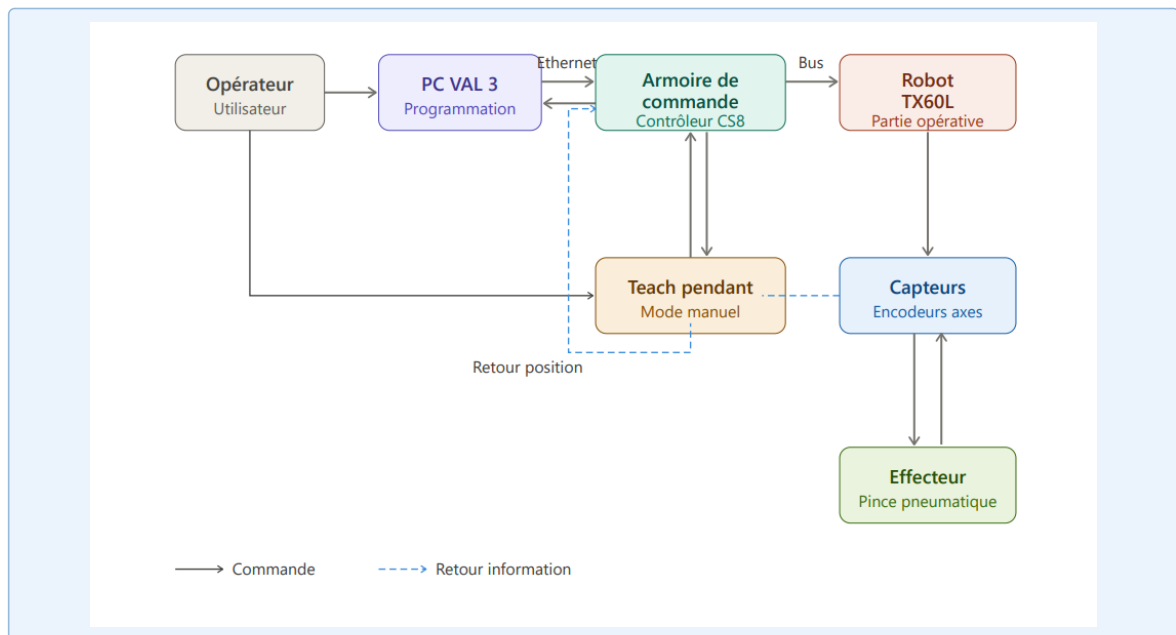


Figure 2.1. Schéma bloc de la cellule robotisée

2.2 Flux informationnels entre les sous-systèmes

Liaison	Description
PC VAL 3 ↔ Armoire	Programme compilé et téléchargé via Ethernet croisé (connecteur J205). L'armoire renvoie les informations d'état au PC.
Teach ↔ Armoire	Commandes manuelles axe par axe transmises en temps réel ; positions courantes affichées en retour.
Armoire → Robot	Consignes de position et de vitesse vers les amplificateurs et les moteurs.
Encodeurs → Armoire	Position angulaire réelle renvoyée en continu pour fermer la boucle.

2.3 Commande d'un axe

La commande de chaque axe repose sur un **asservissement en position en boucle fermée**. Le contrôleur compare la **consigne de position** avec la **position réelle** mesurée par l'encodeur ; l'erreur est traitée par un **correcteur PID** qui génère la commande de courant vers le moteur.

Éléments de la boucle d'asservissement :

- **Calculateur de trajectoire** : génère les consignes de position pour chaque axe selon le programme VAL 3 exécuté.
- **Correcteur PID** : calcule la commande en fonction de l'erreur.
- **Amplificateur de puissance** : convertit le signal en courant moteur.
- **Moteur électrique** : convertit l'énergie électrique en mouvement mécanique.
- **Encodeur** : mesure la position angulaire et la renvoie au contrôleur.

MISE EN ŒUVRE ET PROGRAMMATION

3.1 Procédure de mise en marche du robot

3.1.1 Phase de préparation

1. Vérification que le **champ d'action du robot est libre** de tout obstacle.
2. Mise en position « **On** » de l'interrupteur d'alimentation générale.
3. Branchement de l'**air comprimé** (interrupteur mural en position verticale).
4. Mise en position « **On** » de l'électrovanne ; vérification de la pression entre **1,5 et 2 bars**.

3.1.2 Phase d'utilisation en mode manuel

1. Allumage du **PC et de l'écran**, puis ouverture de session.
2. Mise en position « **On** » de l'interrupteur de l'armoire (les boutons du Teach clignotent = initialisation en cours).
3. Vérification que le **bouton d'arrêt d'urgence est déverrouillé** (tourner et tirer).
4. Soulèvement du Teach, remise sur socle, puis pression sur le **bouton de mise sous puissance**.
5. Sélection du mode **Joint** ou **Frame** pour tester les mouvements.
6. Vérification de l'ouverture/fermeture de la **pince pneumatique** via le bouton jaune (bouton 1).

3.1.3 Phase d'utilisation en mode programmé

1. Vérification des connexions réseau (Ethernet croisé J205, IP : **192.168.0.3**).
2. Vérification de l'adresse IP sur le tableau de bord du contrôleur.
3. Écriture/chargement du programme sous **VAL 3 Studio v6.9.1**, compilation et téléchargement.

4. Lancement de l'exécution depuis l'interface VAL 3.

3.1.4 Phase d'arrêt du robot

1. Quitter les applications sous VAL 3 Studio.
2. Arrêter le PC.
3. Éteindre le Teach via le bouton de mise sous tension.
4. Éteindre l'interrupteur de l'armoire de contrôle.
5. Couper l'alimentation générale par le disjoncteur latéral.
6. Débrancher l'air comprimé.

3.2 Étude de la zone de sécurité et arrêt d'urgence

⚠ Consignes de sécurité :

- **Arrêt d'urgence** (bouton rouge Teach) : coupe immédiatement l'alimentation — usage réservé aux situations de danger réel (arrêt brutal non contrôlé).
- **Arrêt normal** : via le bouton de mise sous tension du Teach, permettant une coupure progressive et contrôlée.
- Le bouton d'arrêt d'urgence doit être **déverrouillé** avant chaque mise sous puissance.

3.3 Prise en main en mode manuel (Teach)

Après la mise en marche, nous avons exploré le robot en **mode manuel** à l'aide du boîtier Teach, en mode **Joint** (axe par axe) ou en mode **Frame** (repère cartésien). Nous avons utilisé **Stäubli 3DStudio** pour visualiser en temps réel la position du robot et les limites articulaires. Nous avons également testé l'ouverture et la fermeture de la **pince pneumatique** (bouton 1), après vérification de la pression entre 1,5 et 2 bars.

3.4 Relevé des coordonnées cartésiennes

Pt.	X (mm)	Y (mm)	Z (mm)	Rx (°)	Ry (°)	Rz (°)	Rôle
p1	694,86	33,85	-120,88	-2,4	165,98	-89,68	Approche prise (zone A)
p2	694,86	33,86	-73,97	-2,4	165,97	-89,68	Prise de la pièce (zone A)
p3	695,05	29,54	-118,84	175,2	14,52	78,56	Approche dépôt (zone B)
p4	143,9	-518,48	-149,53	-178,79	1,85	-2,54	Dépôt (zone B)

Tableau 1 : Coordonnées cartésiennes des quatre points — VAL 3 Studio

3.5 Test d'un programme simple en mode automatique

Avant le cycle complet, nous avons réalisé un **programme de test** pour valider la communication et le comportement du robot en mode automatique.

```
1 begin
2   jactu = herej()
3   j0 = jactu
4   j0.j1 = j0.j1 - 10
5   j0.j2 = j0.j2 + 10
6   open(pince)
7
8   movej(p2, pince, vslow)
9   delay(2)
10  movej(appro(p2, {50, 0, 0, 0, 0, 0}), pince, vslow)
11  delay(2)
12  close(pince)
13 end
```

3.6 Programme final de transfert $A \rightarrow B$ en VAL 3

Une fois les coordonnées validées, nous avons développé le programme final. Deux boucles **for** imbriquées transfèrent une grille de **4 lignes** $\times$ **5 colonnes** (20 pièces) depuis la zone A vers la zone B, avec décalages calculés automatiquement.

```
1 begin
2   for i = 0 to 3 step 1
3     for j = 0 to 4 step 1
4       movej(appro(p2, {j*50, i*50, 0, 0, 0, 0}), pince,
5             vslow)
6       waitEndMove()
7       delay(0.5)
8       movej(appro(p1, {j*50, i*50, 0, 0, 0, 0}), pince,
9             vslow)
10      delay(0.5)
11      close(pince)
12      delay(0.5)
13      movej(appro(p2, {j*50, i*50, 0, 0, 0, 0}), pince,
14            vslow)
15      waitEndMove()
16      delay(0.5)
17      movej(appro(p3, {-j*50, i*50, 0, 0, 0, 0}), pince,
18            vslow)
19      waitEndMove()
20      movej(appro(p4, {-j*50, i*50, 0, 0, 0, 0}), pince,
21            vslow)
22      waitEndMove()
23      open(pince)
24      delay(0.5)
25      movej(appro(p3, {-j*50, i*50, 0, 0, 0, 0}), pince,
26            vslow)
27      waitEndMove()
28    endFor
29  endFor
30 end
```

Les éléments clés du programme sont les suivants :

Instruction	Description
<code>appro(p, {dx,dy,...})</code>	Position décalée par rapport à un point de référence — permet de parcourir toute la grille sans redéfinir chaque point.
<code>waitEndMove()</code>	Attend la fin complète du mouvement avant l’instruction suivante.
<code>for i / for j</code>	4 lignes $\times$ 5 colonnes = 20 pièces au total.
<code>delay(0.5)</code>	Temporisation de 0,5 s pour laisser la pince s’actionner.

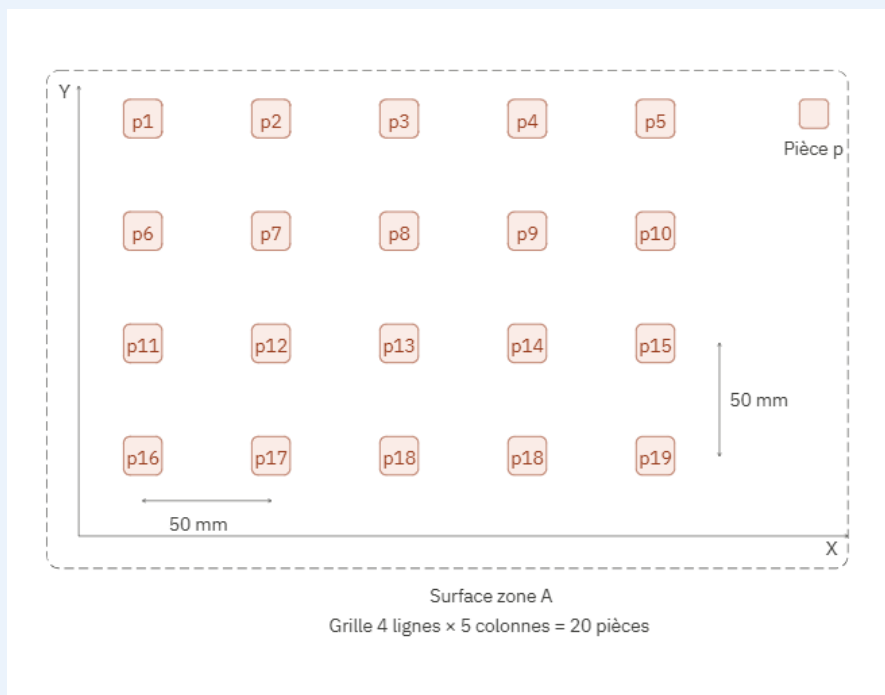


Figure 3.1. Zone A — Points de départ (grille 4×5 , espacement 50 mm)



Figure 3.2. Zone **B** — Points d'arrivée (grille 4×5 , espacement 50 mm)

3.7 Visualisation de la trajectoire calculée

La figure ci-dessous illustre la trajectoire générée par les boucles `for` et l'instruction `appro()`. On y distingue les mouvements verticaux de mise en position et la translation entre les deux zones de travail.

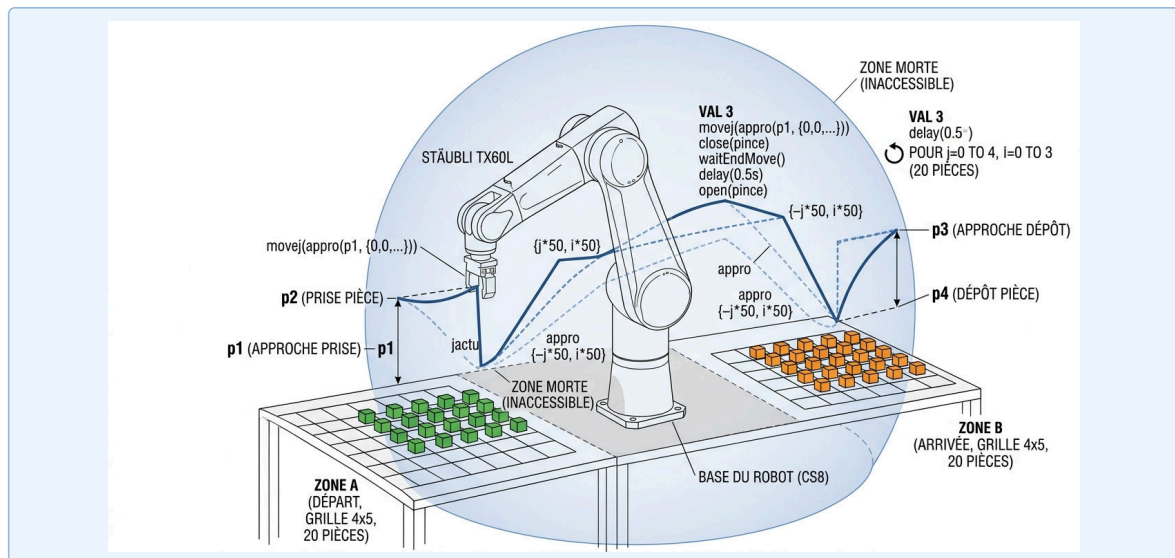


Figure 3.3. Schéma cinématique de la trajectoire $A \rightarrow B$ (points $p1$ à $p4$)

RÉSULTATS ET OBSERVATIONS

4.1 Comportement du robot lors de l'exécution

Lors de l'exécution du programme final, le robot Stäubli TX60L a accompli l'ensemble du cycle de transfert de manière **fluide et répétable**. Plusieurs observations ont été faites tout au long de la manipulation :

- **Précision des mouvements** : le robot a atteint chaque point de référence (p1 à p4) avec une grande précision, confirmant la répétabilité annoncée de $\pm 0,02$ mm. Aucune dérive de position n'a été observée au fil des cycles.
- **Fonctionnement de la pince pneumatique** : l'ouverture et la fermeture de la pince se sont effectuées correctement à chaque cycle. Les temporisations de `delay(0.5)` se sont avérées suffisantes pour permettre à la pince de se mettre en position avant le déplacement suivant.
- **Trajectoires et vitesse** : les mouvements réalisés en mode `vslow` ont permis de sécuriser les approches, notamment lors de la descente sur les pièces. Aucune collision ni dépassement de limite articulaire n'a été constaté.
- **Enchaînement des cycles** : les deux boucles `for` imbriquées ont correctement parcouru l'ensemble de la grille 4×5 , transférant les **20 pièces** une par une de la zone **A** vers la zone **B** sans interruption.
- **Synchronisation** : l'instruction `waitEndMove()` a joué un rôle essentiel en garantissant que chaque mouvement était entièrement terminé avant l'action suivante, évitant ainsi tout décalage dans l'exécution du programme.

✓ **Résultat global** : le cycle de transfert s'est exécuté avec succès pour l'ensemble des 20 pièces. Le temps de cycle par pièce était estimé à environ **8 à 10 secondes** en mode `vslow`, ce qui est cohérent avec les temporisations programmées et la distance parcourue entre les zones A et B.

4.2 Difficultés rencontrées et solutions apportées

Au cours du TP, nous avons rencontré plusieurs difficultés d'ordre technique et logiciel, que nous avons résolues méthodiquement. Le tableau suivant résume les principaux obstacles et les actions correctives mises en œuvre.

Difficulté rencontrée	Solution apportée
Le programme chargé depuis le PC n'est pas exécuté; le Teach affiche systématiquement « programme stocké dans l'historique » et exécute une ancienne version.	Chargement explicite depuis le Teach : après le téléchargement depuis VAL 3 Studio, il est nécessaire de sélectionner manuellement le programme dans l'historique du Teach et de confirmer son exécution. L'habitude a été prise de vider l'historique avant chaque nouvelle session.
Collisions répétées lors de l'exécution du programme, alors que le code VAL 3 est syntaxiquement et logiquement correct. Le robot percute les montants de la cellule ou la table.	Réinitialisation du repère de référence (Teach) : les collisions étaient dues à un décalage progressif du repère de travail, probablement causé par une erreur d'initialisation après un arrêt d'urgence. La solution a consisté à réinitialiser le contrôleur (cycle power off/on) et à recaler les axes via la procédure « Calibration » du Teach avant de relancer le programme.
Problème de communication réseau entre le PC et l'armoire CS8 (échec du téléchargement).	Vérification et correction de l'adresse IP dans VAL 3 Studio (cible : 192.168.0.3) et re-branchement du câble Ethernet croisé sur le connecteur J205.
La pince ne se fermait pas complètement sur la pièce	Augmentation de la temporisation <code>delay()</code> de 0,3 s à 0,5 s pour laisser le temps à l'électrovanne de commuter.
Dépassement de limite articulaire sur l'axe J4	Modification de la trajectoire d'approche en ajustant le décalage dans <code>appro()</code> pour éviter la configuration singulière.

CONCLUSION

Bilan du TP

Ce travail pratique nous a permis d'aborder de façon concrète l'ensemble du cycle de vie d'un système robotisé industriel : de l'analyse structurelle de la cellule à la programmation et l'exécution d'une tâche de manipulation réelle. La mise en œuvre du robot **Stäubli TX60L** nous a confrontés aux réalités du terrain, notamment la gestion des erreurs de communication, le réglage des temporisations et la précision du relevé de points.

Compétences acquises

- **Analyse d'un système automatisé** : identification des sous-systèmes, des flux informationnels et de la boucle d'asservissement en position.
- **Prise en main du robot** : procédures de mise en marche et d'arrêt, navigation en mode Joint et Frame via le boîtier Teach.
- **Programmation VAL 3** : utilisation des instructions de mouvement (`movej`, `appro`), de synchronisation (`waitEndMove`), de temporisation (`delay`) et de contrôle de la pince (`open`, `close`).
- **Débogage et résolution de problèmes** : diagnostic des erreurs réseau, articulaires et de programmation rencontrées lors du TP.
- **Rigueur et sécurité** : respect des procédures de sécurité (arrêt d'urgence, vérification des zones de travail) avant chaque manipulation.

Améliorations possibles

Plusieurs pistes d'amélioration pourraient être envisagées pour des versions futures de ce TP ou dans un contexte industriel réel :

- **Augmentation de la vitesse** : remplacer le mode `vslow` par une vitesse adaptée (`vmax` avec rampe d'accélération) pour réduire le temps de cycle tout en maintenant la sécurité.
- **Détection de présence** : intégrer un capteur (optique ou de force) sur la pince pour confirmer la saisie effective de chaque pièce avant le déplacement.

- **Gestion des erreurs** : ajouter des blocs de traitement d'exceptions en VAL3 pour gérer les cas de pièce manquante ou de défaut pneumatique sans arrêt du cycle complet.
- **Interface de supervision** : développer une interface graphique de suivi en temps réel (état de la pince, numéro de pièce en cours, avancement de la grille).
- **Programmation hors-ligne** : utiliser un logiciel de simulation (Stäubli Robotics Suite) pour préparer et valider les programmes sans mobiliser la cellule physique.